DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Choose the Next Statement

CMSC 124 Activity 12

Prepared by: Rene Andre Bedonia Jocsing

Published: October 22, 2026 **Deadline:** October 25, 2026

Control flow is the repeated decision of which statement executes next. Today one small array will pass through selection, iteration, `continue`, `break`, and fall-through. Keep a next-statement trace, and none of those jumps will have to be guessed.

This activity uses Java snippets. Its rules settle which `if` owns an `else` and how a colon-style `switch` falls through. You aren't expected to know Java beyond what's on the page. Run each example and checkpoint independently. An uncaught exception ends that run. Begin the next checkpoint with its stated initial values.

Laboratory Activity 3 asks you to decide which values your evaluator treats as true. Laboratory Activity 5 uses that rule for control flow.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 16 points. Nine points reward control-flow constructions, and seven reward explanations.

`int[]` declares an array of integers. Array indices start at zero, so an array of length 3 has valid indices 0, 1, and 2. `.length` gives its number of elements. `++` adds one, and `label += "-one"` appends that text to the string `label`. A test such as `score > 6` produces a Boolean value, `true` or `false`.

The short-circuit operator `&&` skips its right operand when the left operand is false. Boolean `&` evaluates both operands unless evaluation throws an exception first. Both use the same **truth table**, the Boolean result for each pair of Boolean inputs. They differ in which operands they evaluate.

The unlabeled `continue` statements here skip the rest of the current loop body. In these for-each loops, execution then requests the next element. Unlabeled `break` exits the nearest enclosing loop or switch. An `else` belongs to the nearest unmatched `if`. In the colon-style switches here, execution **falls through** into the next statement group unless a transfer such as `break` stops it.

Trace the Successor

The **successor** of a statement is the statement that runs next. Trace this complete case by recording each decision or action, its effect on the program state, and its successor:

```
int[] scores = {4, -1, 7};
int index = 0;
boolean readable = index < scores.length && scores[index] >= 0;

int count = 0;
for (int score : scores) {
    if (score < 0) continue;
    if (score > 6) break;
    count++;
```

```
10  }
11
12  String label = "pending";
13  if (count > 0)
14      if (scores[0] > 3)
15          label = "hot";
16      else
17          label = "mild";
18
19  switch (count) {
20      case 0: label += "-none";
21      case 1: label += "-one";
22      default: label += "-done";
23  }
```

The leftmost column gives the line number in the code above, counting blank lines. The trace starts at line 3. Separate actions on the same source line repeat its number. Read each row from left to right. The final column keeps the successor visible.

Line	Phase or input	Executed test or action	Resulting state	Next statement
3	guard	`0 < 3` is true	no change	evaluate `scores[0] >= 0`
3	guard	`4 >= 0` is true	no change	assign `readable`
3	assignment	`readable = true`	`readable = true`	initialize `count`
5	entry	`count = 0`	`count = 0`	load first element
6	iteration	load 4 into `score`	`score = 4`	test `score < 0`
7	score 4	`4 < 0` is false	no change	test `score > 6`
8	score 4	`4 > 6` is false	no change	`count++`
9	score 4	`count++`	`count = 1`	request next element
6	iteration	load -1 into `score`	`score = -1`	test `score < 0`
7	score -1	`-1 < 0` is true	no change	`continue`
7	score -1	`continue`	`count = 1`	request next element
6	iteration	load 7 into `score`	`score = 7`	test `score < 0`
7	score 7	`7 < 0` is false	no change	test `score > 6`
8	score 7	`7 > 6` is true	no change	`break`
8	score 7	`break`	`count = 1`	initialize `label`
12	after loop	`label = "pending"`	`label = "pending"`	test `count > 0`
13	outer `if`	`1 > 0` is true	no change	test `scores[0] > 3`
14	inner `if`	`4 > 3` is true	no change	`label = "hot"`
15	assignment	`label = "hot"`	`label = "hot"`	evaluate `switch (count)`
19	switch	`count` is 1: select `case 1`	no change	`label += "-one"`
21	`case 1`	`label += "-one"`	`label = "hot-one"`	`label += "-done"`
22	`default`	`label += "-done"`	`label = "hot-one-done"`	leave switch

The trace follows the written program. `continue` starts the next iteration, `break` leaves the loop, the `else` belongs to the inner test, and case 1 falls through to `default`.

Start at the Boundary

The short-circuit operator `&&` evaluates its right operand only when the left operand is true. Boolean `&` evaluates both operands unless an earlier operand throws an exception. Both combine Boolean results.

Here, only `&&` skips the unsafe access when the index equals the array length. An upper-bound test alone would not protect a negative index.

Worked Example for Checkpoint 1

Suggested time: 16 minutes, including the worked example.

For `int[] one = {8}; int i = 1;`, `i < one.length` is false. In `i < one.length && one[i] > 0`, conditional `&&` skips the array access and returns false. Replacing it with `&` forces the access to `one[1]`. That index is outside the array.

Read `&&` as "conditional and" and `&` as "eager Boolean and."

Stopping after observing that both operators share a truth table misses the exception before eager `&` can combine two Boolean results.

Checkpoint 1: Choose or Skip (4 Points)

Running Case

```
int[] temperatures = {18, -1, 27};
int index = 3;
boolean readable = index < temperatures.length &&
                   temperatures[index] >= 0;
```

1. (Construction: 2 points) Trace the initializer of `readable`, including the operand that runs and the skipped operand. Then replace `&&` with `&` and trace until evaluation produces a result or an exception.

2. (Explanation: 2 points) Explain why the two operators have the same truth table but different behavior here. Identify the boundary value that makes the difference.

Carry the Array Through a Loop

A **for-each loop** requests the next element, assigns it to the loop variable, runs the body, and repeats while another element exists.

In this loop, `continue` requests the next element and `break` leaves the loop. The nested `if` and the colon-style `switch` then determine which assignments run.

Worked Example for Checkpoint 2

Suggested time: 30 minutes, including the worked example.

Return to the model with scores `{4, -1, 7}`. Each row records one executed test, assignment, element load, or transfer. A true test leads to a separate `continue` or `break` row. A false test leads to the next test or assignment. Use that same detail below.

Checkpoint 2: Name Every Successor (7 Points)

Running Case

Start a fresh run with the same array, `temperatures = {18, -1, 27}`:

```
int[] temperatures = {18, -1, 27};
int count = 0;

for (int temperature : temperatures) {
    if (temperature < 0) continue;
    if (temperature > 25) break;
    count++;
}
```

```

String label = "pending";
if (count > 0)
    if (temperatures[0] > 20)
        label = "hot";
    else
        label = "mild";

switch (count) {
    case 0: label += "-none";
    case 1: label += "-one";
    default: label += "-done";
}
```

1. (Construction: 4 points) Use the model's five columns to trace one control-flow decision or action per row. Continue through the nested `if` and `switch`. Give the final `label`.

2. (Explanation: 3 points) Answer all three. Each answer is worth one point.

a. Why doesn't `-1` increment `count`, and why does `27` end the loop?
b. Which `if` owns the `else`, and which assignment therefore runs?
c. Why do both `case 1` and `default` add text to `label`?

Preserve the Loop's Path

We can rewrite this array's for-each loop as a counter-controlled `while` loop. Its index stores the next position to load. The rewrite must preserve the element visits and the effects of `continue` and `break`. It must also terminate.

This is **desugaring**, translating a convenient construct into simpler constructs with the same behavior. Laboratory Activity 5 uses the same idea for C-style `for` loops in your language. There, the parser can build a `while` syntax tree for your evaluator to execute. Today's example translates a Java array loop directly into Java statements.

Worked Example for Checkpoint 3

Suggested time: 14 minutes, including the worked example.

Consider this for-each loop that uses `continue`:

```
int[] sample = {5, -2, 8};
int seen = 0;

for (int value : sample) {
    if (value < 0) continue;
    seen++;
}
```

This equivalent counter-controlled loop preserves progress even when `continue` runs:

```
int[] sample = {5, -2, 8};
int i = 0;
int seen = 0;

while (i < sample.length) {
    int value = sample[i];
    i++;
    if (value < 0) continue;
```

```
    seen++;
}
```

The update follows the load and precedes any `continue`. Track it through the negative element:

Index before load	Current element	Index after update	Action	`seen` afterward
0	5	1	increment `seen`	1
1	-2	2	`continue`	1
2	8	3	increment `seen`	2

The next guard, `3 < 3`, is false. No element is loaded at index 3. A bottom-only increment would be skipped by `continue`, leaving the loop on the same negative element forever.

You may instead increment immediately before each `continue` and at the end of the normal path. Every path that returns to the guard must advance once. A path that exits through `break` needn't advance because it never returns to the guard.

Checkpoint 3: Preserve the Path (5 Points)

Convert this complete source loop for `temperatures = {18, -1, 24}`:

```
int count = 0;
for (int temperature : temperatures) {
    if (temperature < 0) continue;
    if (temperature > 25) break;
    count++;
}
```

The `while` version must visit the same elements in order and preserve the same `continue` and `break` effects. Advance the index once per iteration that returns to the guard. Stay within the array bounds and finish with the same `count`. Keep the `break` test even though this input never takes it.

1. (Construction: 3 points) Write the `while` version and trace the index, current element, and `count` through termination.

2. (Explanation: 2 points) Explain where the index update must occur so `continue` still makes progress. State why placing it only at the bottom of the body would fail.

Equivalent loops receive full credit when they preserve the visited elements, their order, and the loop transfers. They must stay within bounds and finish with the same `count`. Matching the count alone is insufficient. End with the transfer trace as a comment beside the line or lines that guarantee progress.